



UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR



**Bsc Electrical Engineering
Department of Electrical Engineering**

LAB NOTEBOOK

Programming Language C++

**SUBMITTED TO:
ENGR.MUHAMMAD RIZWAN**

EE-170L COMPUTER PROGRAMMING LAB

Table Content

S.no	Table of contents	Page no
	Student Info	01
1	Installation of C++	02
2	Data type , memory size and memory address in C++	06
3	Comparison,logical and condition operation in C++	08
4	Switch Statment	11
5	While loop in C++	14
6	The for loop in C++	18
7	ONE dimensional Array in C++	21
8	Two dimensional Array in C++	24
9	Introduction to function in C++	28
10	More about function in C++	32
11	String in C++	35
12	Pointers in C++	39
13	Structures in C++	43
14	Open Ended Lab (OEL)	52

Student Name:

Muhammad Sahal Ayub

Etea Id:

4042

Section :

B

Cousre code:

EE-170L COMPUTER PROGRAMMING LAB

Programming language:

C++

Subject:

Computer programming Lab

Department:

Electrical Engineering

Field :

Computing and Ai

Professer:

Engr Muhammad Rizwan

Session:

Spirng 2026

Lab 01

Installation of C++

LAB TITLE:

INSTALLATION OF CODE BLOCKS

OBJECTIVES:

Introduction Of C++ Programming & How to Install Code Blocks for C++ on Windows?

TOOLS:

- PC
- INTERNET
- BROWSER
- DOWNLOADER

INTRODUCTION:

C++ (or “C-plus-plus”) is a high-level, general-purpose programming language designed for system and application programming. It was developed by Bjarne Stroustrup at Bell Labs in 1983 as an extension of the C programming language. C++ is an object-oriented, multi-paradigm language that supports procedural, functional, and generic programming styles.

C++ is used in developing browsers, operating systems, and applications, as well as in-game programming, software engineering, data structures, etc.

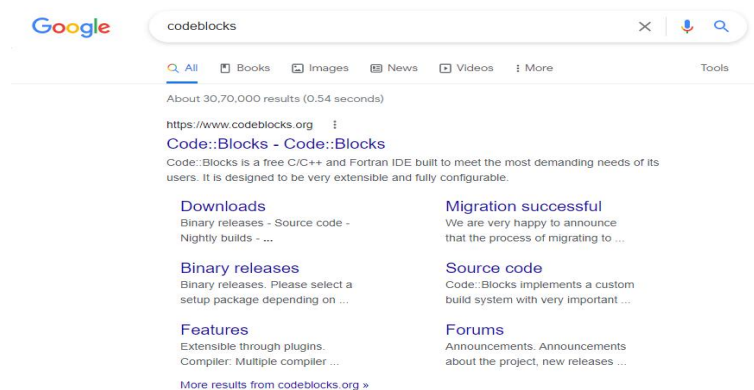
PROCEDURES:

Follow the below steps to install Code Blocks for C++ on windows:

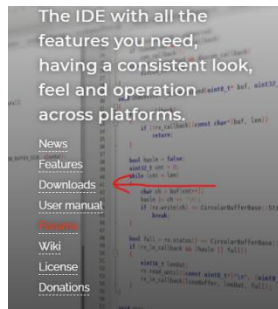
Step 1: Open Your Web Browser

Step 2: Go to the Search Panel and Search for “Code Blocks”

Step 3: Click on the First Result shown on the Search Engine.



Step 4: Click on the “Downloads” Section.



Code::Blocks

The free C/C++ and Fortran IDE

Code::Blocks is a free C/C++ and Fortran IDE designed to be very extensible and fully configurable.

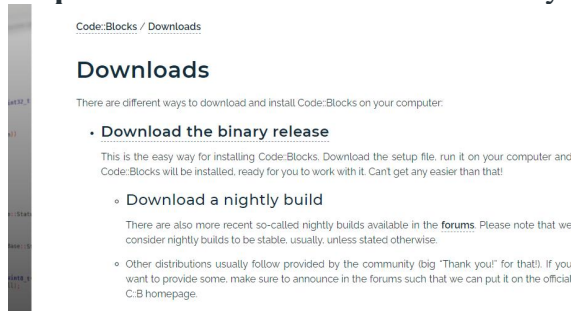
Built around a plugin framework, Code::Blocks is added by installing/coding a plugin. For instance, by plugins!

If you're new here, you can read the [user manual](#) and join our [forums](#) to find help or general discussion.

We hope you enjoy using Code::Blocks!

The Code::Blocks Team

Step 5: Click On “Download the binary release”.



Step 6: As per this Date, the Latest Version of Code Blocks is 20.03. Now here you'll find several download options

1. codeblocks-20.03-setup.exe : It is a standard executable file that is easier to install.
2. codeblocks-20.03-nosetup.zip : It is a zip file which you do not have to install , you can just unzip and run it.

I'll suggest you download the file with MinGW written on it (“codeblocks-20.03mingw-setup.exe”) as MinGW is a Compiler that is needed to run the Program. If you download the normal setup file then you have to download The compiler separately.

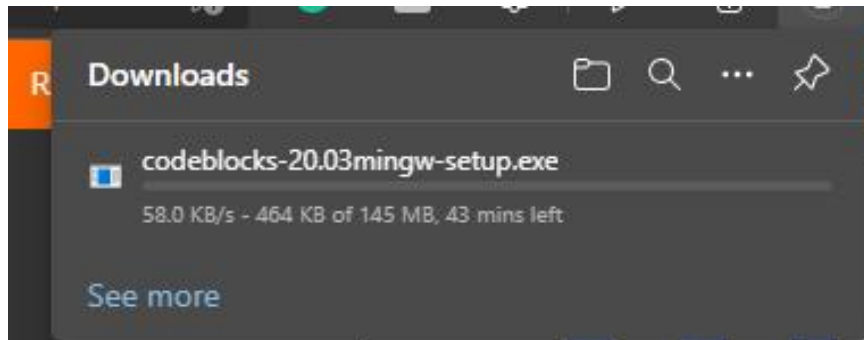


File	Download from
codeblocks-20.03-setup.exe	FossHUB or Sourceforge.net
codeblocks-20.03-setup-nonadmin.exe	FossHUB or Sourceforge.net
codeblocks-20.03-nosetup.zip	FossHUB or Sourceforge.net
 codeblocks-20.03mingw-setup.exe	FossHUB or Sourceforge.net
codeblocks-20.03mingw-nosetup.zip	FossHUB or Sourceforge.net
codeblocks-20.03-32bit-setup.exe	FossHUB or Sourceforge.net
codeblocks-20.03-32bit-setup-nonadmin.exe	FossHUB or Sourceforge.net
codeblocks-20.03-32bit-nosetup.zip	FossHUB or Sourceforge.net
codeblocks-20.03mingw-32bit-setup.exe	FossHUB or Sourceforge.net
codeblocks-20.03mingw-32bit-nosetup.zip	FossHUB or Sourceforge.net

NOTE: The codeblocks-20.03-setup.exe file includes Code::Blocks with all plugins. The codeblo setup-nonadmin.exe file is provided for convenience to users that do not have administrator rights on their machine(s).

Step 7: Click on Sourceforge.net under the Download Section of your desired file.

Step 8: then Download should begin within some seconds.

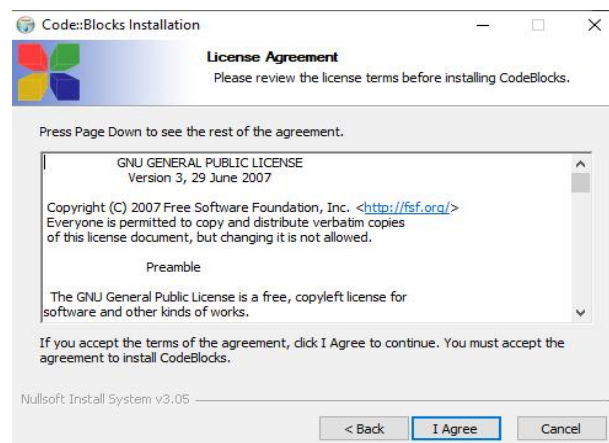
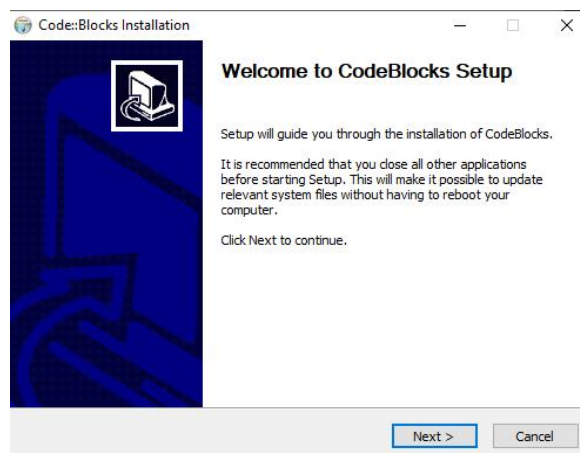


Step 9: When the download is completed, Open Your Code Blocks Setup File.

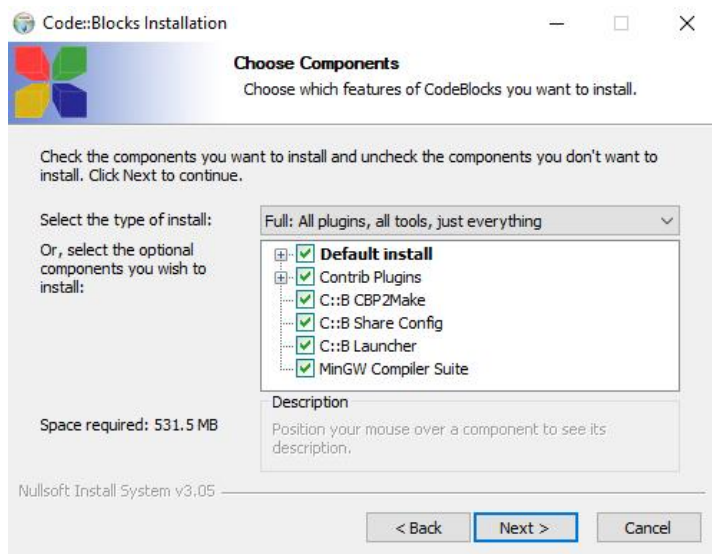
Step 10: Click on next.

Step 11: Click on I agree.

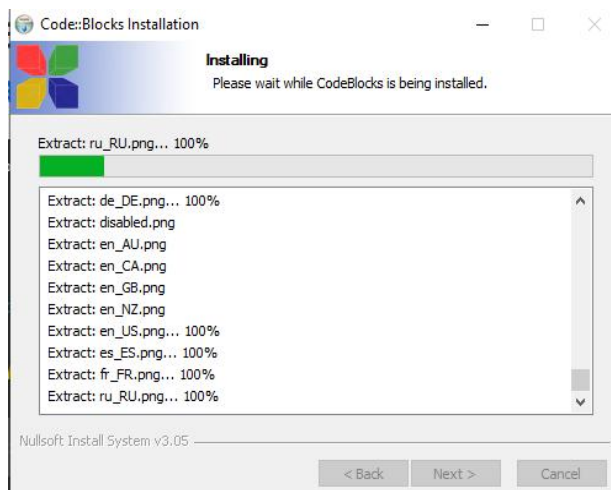
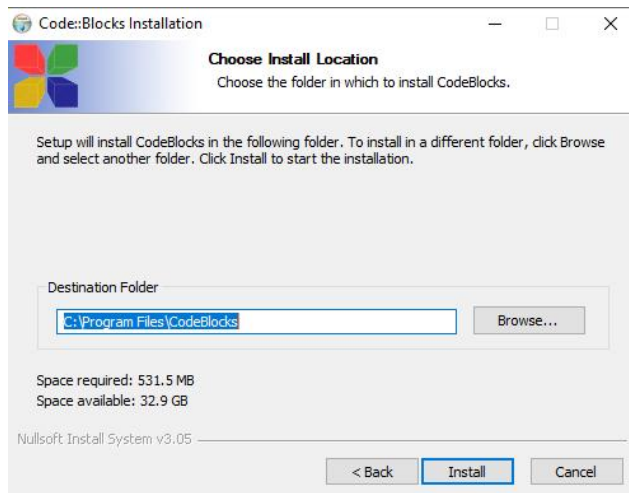
Step 11: Click on I agree.



Step 12: Click On Next (you need to have at least 600 MB free storage on your drive for the installation).



Step 13: Select your Destination and Click on Install



Step 14: Once Installation gets completed, click on Next and then Click on Finish

Now You CodeBlocks have been installed.

Observation:

Hence We observed that how to Download the Code Blocks From the Internet using the browser and how to Install It In Our Pc or windows .

Lab 02

Data type Memory size and memory address

Learning Objectives:

The objective of this lab is:

- To understand different data types in C++
 - To learn how to declare variables
 - To observe the memory size of different data types
- To find the memory addresses of variables using the address operator (&)

Theory:

In C++, variables are used to store data in memory.

Each variable has:

- Data Type – defines what kind of data it can store.
- Memory Size – the amount of memory allocated for the variable.
- Memory Address – the location where the variable is stored

Data Type	Description
int	Stores integer numbers
float	Stores decimal numbers
double	Stores large decimal
char	Stores a single character
bool	Stores true or false

Algorithm:

1. Start the program
2. Declare variables of different data types
3. Assign values to variables
4. Print the values of variables
5. Use sizeof() to print memory size
6. Use & operator to print memory addresses

End the program

Program Code:

```
#include <iostream>
using namespace std; //
Int main(){
```

```
int a=10;
Float b=5.5;
Double c=20.99;
Char d='A';
Bool e =true;

cout<<'VALUES:\n';
cout<<"int a="<<a<<endl;
cout<<"float b=" <<b<< endl;
cout <<"double c="<<c<<endl;
cout<<"char d=" <<d<< endl;
cout<<"bool e=" <<e<< endl;

cout<<"\nMEMORYSIZE(in bytes):\n=";
cout<<"size of float:" <<size of(c)<< endl; //endl is used for next line
cout<<"size of int " <<size of (b)<< endl;
cout<<"size of double" <<size of (c)<< endl;
cout<<"size of Char" <<size of (d)<< endl;
cout<<"size of bool" <<size of(e)<< endl;

cout<<"\nMEMORY ADDRESS(LOCATION):\n"; //this show memory location
cout<<"Address of a:" <<&a<< endl;
cout<<"Address of b:" <<&b<< endl;
cout<<"Address of c:" <<&c<< endl;
cout<<"Address of d:" <<&d<< endl;
cout<<"Address of e:" <<&e<< endl;

Return 0;
}
```

Program Output:

```
Before: x=50, y=70
Values:
Int a=10
Float b=5.5
Double c=20.99
Char d=A
Bool e=1
```

```
Memory size (in bytes):
Size of int=4
Size of float=4
Size of double=8
Size of char=1
Size of bool=1
```

```
MEMORY ADDRESS(LOCATION)
Address of a:0x61ff08
Address of b:0x61ff04
Address of c:0x61ff08
Address of d:0x61ff07
Address of e:0x61ff06
```

Result:

The program successfully displayed;

- The program successfully displayed:
- Values of different variables
- Memory size of each data type
- Memory addresses of variables

Conclusion:

From those experiment we learned;

- Different **C++ data types store different kinds of data**
- Each data type occupies **different memory sizes**
- The **address operator (&)** can be used to find the memory location of variables

Lab 03

Comparison , Logical and Condition Operation in C++

Learning Objectives:

The objective of this lab is to practice the use of:

Comparison/Relational operators

Logical operators

Conditional statements: if, if-else, and else-if in C programming

Theory:

1. Comparison /Relational operator:

Comparison operators are used to compare two values in C. They return either true (non-zero) or false (0).

== : Equal to

!= : Not equal to

> : Greater than

< : Less than

>= : Greater than or equal to

<= : Less than or equal to.

Example:

```
If (a>b) {  
    cout<<"a is greater than b";  
}
```

2. Logical Operators:

Logical operators are used to combine multiple conditions:

&& : Logical AND (true if both conditions are true)

|| : Logical OR (true if at least one condition is true)

! : Logical NOT (inverts the result)

Example:

```
If (a>0 && a<100) {  
    cout<<"Number is between 1 and 99";  
}
```

3. Conditional Statements:

Conditional statements control the flow of the program based on conditions.

a) if statement – executes code block if condition is true:

```
If (a>0 ) {  
    cout<<"Positive number";  
}
```

```
}
```

b) if-else statement – provides two paths:

```
If(age>=18 ) {  
    cout<<"Positive number";  
Else  
Cout <<"not eligible";  
}
```

c) else-if statement – handles multiple conditions:

```
If(a>=0){  
    cout<<"Positive number";  
Else if(a<0)  
Cout <<"Negative";  
Else  
cout<<"zero";  
}
```

Task 1:

Comparison/Relational Operators

Description:

Compare two numbers and display the result.

Code:

```
#include <iostream>  
Using namespace std;  
Int main(){  
    Int num1,num2;  
    Cout<<"enter frist number:";  
    cin>> num1;  
    Cout <<"enter second number";  
    Cin>>num2;  
    If(num1==num2)  
    Cout<< "both number are equal."<<endl;  
    Else if(num1>num2)  
    Cout<<"first number is greater than second number."<<endl;  
    Else  
    Cout <<"frist number os less than second."<<endl;  
    return 0;  
}
```

Output:

```
Enter an integer :50  
Number is greater than 1 and less than 100.
```

Task 2:

Logical Operators

Description: Check if a number is greater than 0 and less than 100.

Code:

```
#include <iostream>  
Using namespace std;  
Int main(){  
    Int  
    cout<<"enter frist number:";  
    cin>>num;  
    if (num >0&& num<100)  
    cout <<"number is greater than 0 and less than 100.<<endl;  
    else  
    cout<<"number doesnot meet the conditons."<<endl;  
    return0;
```

```
}
```

Output:

```
Enter an integer :50
Number is greater than 1 and less than 100.
```

Task 03:

If Statement:

Description: Check if a number is positive.

Code:

```
#include <iostream>
Using namespace std;
Int main(){
    Int ;
    cout<<"enter an integer:";
    cin>>num;
    if (num >0)
        cout <<"The number is postive".<<endl;
    else
        cout<<"The number is not positive."<<endl;
    return 0;
}
```

Output:

```
Enter an integer :-5
The number is not positive.
```

Task 04:

If-else Statement:

Description: Check voting eligibility (age ≥ 18).

Code:

```
#include <iostream>
Using namespace std;
Int main(){
    Int age;
    cout<<"enter your age:";
    cin>>age;
    if (age >=18)
        cout <<"you are eligible to vote".<<endl;
    else
        cout<<"you are not eligible to vote."<<endl;
    return 0;
}
```

Output:

```
Enter your age:20
You are not eligible for vote
```

Task 05:

Else-if Statement:

Description: Check if a number is positive, negative, or zero.

Code:

```
#include <iostream>
Using namespace std;
Int main(){
    Int num;
```

```
cout<<"enter an integer:";
cin>>num;
if (num >0)
cout <<"The number is postive".<<endl;
else if(num<0)
cout<<"The number is not positive."<<endl;
return0;
}
```

Output:

```
Enter an integer:0
The number is zero.
```

Conclusion:

- Learned comparison and logical operators in C.
- Practiced conditional statements (if, if-else, else-if).
- Understood how programs make decisions based

Gained experience in structuring programs logically to solve problems.

Lab 04

Switch Statement in C++

Learning Objectives:

- Learn and practice switch statements in C++.
- Implement menu-driven programs and case-based decision making.
- Understand how break statements prevent fall-through in switch.
- Apply switch statements to solve real-world programming problems

Theory:

Switch Statement:

A switch statement allows a variable to be tested for equality against a list of values (cases).

Syntax:

```
Switch (expression) {
    case value1:
        //statements
        break;
    case value 2:
        //statements
        break;
    ...
    default:
        //statement if no case matches
}
```

Key points:

- Each **case** ends with a break statement to prevent fall-through.
- **default** executes if no cases match.
- **switch** works with integers, characters, or enums, but not with floats or strings.

Example:

```
#include <iostream>
Char grade ='b';
Switch(grade){
Case 'a':cout<<"excellent";break;
Case 'B':cout<<"very good";break;
Default:cout<<"invalid grade";
}
```

Task 1:

Day name using switch statement

Description:

Program takes a number (1–7) and prints the corresponding day name.

Code:

```
#include <iostream>
Using namespace std;
Int main(){
    Int day;
    cout<<"enter day number (1-7):";
    cin>>day;

    switch (day){
    case 1:cout <<"Monday" ; break;
    case 2:cout <<"Tuesday" ; break;
    case 3:cout <<"Wednesday" ; break;
    case 4:cout <<"Thursday" ; break;
    case 5:cout <<"Friday" ; break;
    case 6:cout <<"Saturday" ; break;
    case 7:cout <<"Sunday" ; break;
    default: cout<<"Invalid day number";
    }
    cout<<endl;
    return 0;
}
```

Output:

```
Enter day number (1-7):3
Wednesday
```

Task 2:

Grade Evaluation Using Switch Statement:

Description: Program takes a character grade (A–F) and prints evaluation.

Code:

```
#include <iostream>
Using namespace std;
Int main()
{
    char grade; //character type variable

    cout<<"enter your grade (A,B,C,D,F)";
```

```
cin>>grade;
switch(grade)          //Grade ki value check karege
{
    case 'A':
        cout<<"Excellent";
        break;
    case 'B':
        cout<<"Very good";
        break;
    case 'C':
        cout<<"good";
        break;
    case 'D':
        cout<<"Pass";
        break;
    case 'F':
        cout<<"Fail";
        break;
    default:           // agar koi aur letter ho
        cout<<"invalid grade.";
}
return 0;
}
```

Output:

```
Enter your grade (A, B, C, D, F):B
Very good
```

Task 3:

Unit Conversion Method Using Switch Statement

Description: Program shows a menu for conversions and performs the selected operation.

Code:

```
#include <iostream>
using namespace std;
int main() {
    int choice;           // Menu choice store karega
    double value;         //input value store
    karega
    double result;        // result store karega

    cout<<"1 ->km to meters "<<endl;
    cout<<"2 ->meter to cm "<<endl;
    cout<<"3->kg to grams "<<endl;
    cout<<"4 ->celsius to fahrenheit "<<endl;
    cout<<"enter your choice:";
    cin>>choice;
    switch(choice)        //user ki choice
    check kargea
    {
        case 1:
            cout<<"Enter kilometers:"<<endl;
            cin>>value;
            result =value *1000;    //km ko meter mein change kare ga
            cout<<"meter = "<<result;
            break;
    }
```

```
        case 1:
cout <<"Enter kilometers:".<<endl;
cin >>value;
    result =value *1000;           //km ko meter mein change kare ga
cout<<"meter = "<<result;
break;

        case 2:
cout <<"Enter meters:".<<endl;
cin >>value;
    result =value *100;           //meter ko cm mein change kare ga
cout<<"Centimeter = "<<result;
break;

        case 3:
cout <<"Enter kilogram:".<<endl;
cin >>value;
    result =value *1000;           //kg ko gram mein change kare ga
cout<<"grams = "<<result;
break;

        case 4:
cout <<"Enter celsius:".<<endl;
cin >>value;
    result =(value *9/5)+32;       //celsius ko fahrenheit mein
change kare ga
cout<<"fahrenheit = "<<result;
break;
default:
    Cout<<"invalid choice";
}
return0;
}
```

Output:

```
Enter
Conversion menu:
1 kilometer to meter
2 meter to centimeters
3 kilogram to grams
4 celsius to fahrenheit
Enter your choice:4
Enter celsius: 25
25 C = 77 F
```

Conclusion:

- Learned how to use switch statements for multiple case-based decisions.
- Understood the importance of break to avoid fall-through.
- Applied switch statements to practical problems like day names, grade evaluation, and unit conversions.
- Learned to create menu-driven programs in C++ for better user interaction.

Lab 05

The While Loop in C++

Learning Objectives:

The objective of this lab is to understand and implement looping structures in C++. The main goals of this lab

are:

- To understand the working of the while loop.
- To learn about nested while loops.
- To implement the do-while loop in C++ programs.
- To perform repetitive calculations using loops.

Theory:

While Loop:

A while loop is a control structure that allows a block of code to execute repeatedly as long as a specified condition remains true. The condition is checked before executing the loop body. If the condition is true, the statements inside the loop are executed. The loop continues until the condition becomes false.

Syntax:

```
while (condition)
{
    //statements
}
```

Nested While Loop:

A nested while loop is a loop inside another loop. The inner loop executes completely for each iteration of the outer loop.

Syntax:

```
while (condition)
{
    while (condition2)
    {
        //statements
    }
}
```

Do-While Loop:

The do-while loop is similar to the while loop but with one important difference.

The loop body executes at least once, because the condition is checked after executing the statements.

Syntax:

```
do
{
    //statements
}
while (condition);
```

Lab Tasks & Programs

Display Number Using The While Loop:

Description:

In this task, a C++ program was developed using a while loop to display numbers from 1 to 10. The loop starts with an initial value and continues to execute until the condition becomes false. Each number is printed on a separate line to demonstrate how the while loop repeatedly executes the statements inside the loop.

Code:

```
do
#include <iostream>
```

```
using namespace std;
int main(){
    int i=1;           //yeha varaible I ko 1 se start keya hai
    while (I<=10)      // jab tak I 10 se chota ya barabar hai loop
    chalta rahega
    {
        cout <<I<<end; // current number screen par print karega
        i++;           // I ki value 1 sw barh jayegi (increment)
    }

    return 0;         //program end
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Task 2:

Display Number Using DO-While Loop:

Description

In this task, the same program was implemented using a do-while loop. This loop executes the program statements first and then checks the condition. This ensures that the loop body runs at least once, even if the condition is false.

Code:

```
#include <iostream>
using namespace std;
int main(){
    int i=1;           //yeha varaible I ko 1 se start keya hai
    do                // jab tak I 10 se chota ya barabar hai loop chalta
    rahega
    {
        cout <<i<<end; // current number screen par print karega
        i++;           // I ki value 1 sw barh jayegi (increment)
    }
    while (i <=10);    //condition check hogi je I 10 se chota
    ya barabar hai ya nahi hai

    return 0;         //program end
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Task 3:

Sum Of Number Using While-Loop:

Description:

In this task, a C++ program was written to calculate the sum of numbers from 1 to a user-entered positive integer. The program takes input from the user and uses a while loop to repeatedly add numbers until the given number is reached. Finally, the calculated sum is displayed.

Code:

```
#include <iostream>
using namespace std;
int main(){
    int num, i=1 , sum =0;
    //num user se number lene ke liye
    // I counting ke liye
    // sum total store karne ke liye
    {
        cout <<"enter a positive integer:";           // user ko message
        show
        cin>>num;           // user se number input lena

        while (i <= num);           //jab tak I user ke number se chota ya
        barabar hai

        sum= sum +i;           //har dafa I ko sum me add karna
        i++;           // I ko 1 barhana
    }
    cout <<"sum ="<<sum;           //final sum display
    return 0;           //program end
}
```

Output:

```
Enter a positive integer:5
Sum =15
```

Result:

In this lab, we successfully implemented while loop, nested loop concept, and do-while loop in C++. We learned how loops help execute repetitive tasks efficiently.

Conclusion:

This lab helped us understand the basic concept of looping structures in C++.

We learned how to use the while loop to execute a set of statements repeatedly based on a condition.

We also studied the do-while loop, which ensures that the loop body executes at least once.

We practiced displaying numbers using loops.

We learned how loops can be used to perform repetitive calculations, such as finding the sum of numbers.

Overall, this lab improved our understanding of loop control and program execution in C++.

Lab 06

The For Loop in C++

Learning Objectives:

The objective of this lab is to understand and apply the for loop in C++. Students will learn how to:

- Use a for loop to repeat a block of code a fixed number of times.
- Use nested for loops to work with patterns and tables.
- Calculate sums using for loops.
- Print patterns using nested for loops.

Introduction:

A for loop is a repetition control structure that allows a programmer to write a loop that executes a specific number of times. It is used when a variable runs from a starting point to an ending point with a constant increment or decrement.

Syntax of for loop:

```
#include <iostream>
For (initialization; condition: update)
{
    //statements
}
```

A nested for loop is a for loop placed inside another for loop. It is commonly used for printing patterns or

Task 1:

For Loop – Display Numbers 1 to 10

Write a C++ program that uses a for loop to display numbers from 1 to 10, each on a separate line.

Code:

```
#included <iostream>
using namespace std;
int main()
{
    int i=1; //yeha varaible i ko 1 se start keya hai
    //Loop from 1 to 10
    while ( <=10)
    {
        cout <<i<<endl; // current number screen par print karega
        i++; // i ki value 1 sw barh jayegi (increment)
    }
    return 0; //program end
}
```

Output:

```
1
2
3
4
```

```
5
6
7
8
9
10
```

Task 2:

Nested for Loop – Multiplication Table :

Write a C++ program that generates a multiplication table from 1 to 10 using nested for loops.

Code:

```
#include <iostream>
using namespace std;
int main()
{
    // row
    for (int i=1; i <=10; i++) //it is for two dimession it
    is used

    //columns
    for (int j=1; j <=10; j++)
    {
        cout << i * j << "\t";
    }
    cout << endl; // for next line
}

return 0; //program end
}
```

Output:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Task 3:

Sum Using for Loop

Write a C++ program that prompts the user to enter a positive integer and uses a for loop to calculate the sum of all numbers from 1 to that integer.

Code:

```
#include <iostream>
using name space std;
Int main()
{

Int    num,sum =0;

Cout  << "enter a positive integer:";
```

```
Cin>>num;

for (int i=1; i <=10; i++) //it is for two dimession it is
used

{
    sum += i;
}
    cout << "Sum ="<< sum << endl;

return 0; //program end
}
```

Output:

```
Enter a postive integer:5
Sum from 1 to 5 = 15
```

Task 4:

Nested Pattern Printing:

Write a C++ program that uses nested for loops to print a triangle pattern of asterisks. The number of rows is determined by user input.

Code:

```
#include <iostream>
using name space std;
Int main()
{

    Int row;

    Cout << "enter number of rows:";

    Cin>> rows;

    for (int i=1; i <=row; i++) //it is for two dimession it is
    used

    {
        For (int j=1; i<=i; j++)
        {
            cout <<"*";
        }
        cout <<endl;
    }

    return 0; //program end
}
```

Output (for rows = 5):

```
*
* *
* * *
* * *
* * *
```

* * * *
* * * * *

Conclusion:

In this lab, we successfully learned and applied the for loop in C++. We used it to display numbers, calculate sums, generate a multiplication table, and print triangle patterns. The nested for loop proved especially useful for working with rows and columns. These exercises helped us understand how loops control program flow and reduce repetitive code

Lab 07

One Dimension Array in C++

Learning Objectives:

The objective of this lab is to understand and apply one-dimensional arrays in C++. Students will learn how to:

- Declare and initialize arrays in C++.
- Access and modify individual elements of an array using indices.
- Copy elements from one array to another using loops.
- Perform arithmetic operations (e.g., sum) on array elements.
- Work with character arrays (strings) in C++.

Introduction:

An array is a data structure in C++ that stores a fixed-size sequential collection of elements of the same type. Arrays are stored in contiguous memory locations, where the lowest address

corresponds to the first element and the highest address to the last element.

To declare an array:

Syntax:

```
Datatype arrayname [arraysize];
```

Example: `int age[4];` declares an integer array of 4 elements.

To initialize an array:

```
int age [4] = {23, 34, 64, 75};
```

Array indices start at 0, so `age[0]` is the first element and `age[3]` is the last.

Task 1:

Array Declaration and Initialization

Declare an integer array called 'numbers' with a size of 5. Initialize it with values 10, 20, 30, 40, 50, and display all elements using a loop.

Code:

```
#include <iostream>
using namespace std;
int main ()
{
    // declare and initialize integer array of size 5
```

```
Int numnber [5] = {10, 20, 30, 40, 50,};  
  
//display array elements using loop  
  
Cout << "array elements are :\n";  
For (int i=0;    i <5;    i++ )  
{  
    //print each element of array  
    Cout <<"number ["<< I <<" ] =" << number    [i]    << endl;  
}  
return 0;  
}
```

Output:

```
Array element are:  
Number[0]=10  
Number[1]=20  
Number[2]=30  
Number[3]=40  
Number[4]=50
```

Task 2:

Accessing Array Elements:

Declare a character array called 'message' to store a word of your choice. Use a loop to access each character and display it on a separate line.

Code:

```
#include <iostream>  
using namespace std;  
int main()  
{  
  
    // Declare character array to store a word  
  
    char message[] = "HELLO";  
  
    // Display each character on separate line  
  
    cout << "Characters of the word are:\n";  
  
    for (int i = 0; message[i] != '\0'; i++)  
    {  
  
        // Print each character until null character  
  
        cout << message[i] << endl;  
  
    }  
    return 0;  
}
```

Output:

```
Characters of the word are:  
H  
E  
L
```

L
O

Task 3:

Copying Arrays:

Declare two integer arrays 'source' and 'destination', each of size 5. Initialize 'source' with values, copy elements to 'destination' using a loop, and display both arrays.

Code:

```
#include <iostream>
using namespace std;
int main()
{
    // Declare source and destination arrays

    int source[5] = {5, 10, 15, 20, 25};

    int destination[5];

    // Copy elements from source to destination

    for (int i = 0; i < 5; i++)
    {
        destination[i] = source[i];
    }
    // Display source array
    cout << "Source Array:\n";
    for (int i = 0; i < 5; i++)
    {
        cout << source[i] << " ";
    }
    cout << endl;

    cout << "Destination Array:\n";

    for (int i = 0; i < 5; i++)
    {
        cout << destination[i] << " ";
    }
    return 0;
}
```

Output:

```
Source Array:
5 10 15 20 25
Destination Array:
5 10 15 20 25
```

Task 4:

Array Sum

Write a C++ program that prompts the user to enter 5 numbers, stores them in an array, calculates

the sum of all numbers, and displays the result.

Code:

```
#include <iostream>
using namespace std;
int main()
{
    // Declare array for 5 numbers

    int numbers[5];
    int sum = 0;

    // Take input from user
    cout << "Enter 5 numbers:\n";

    for (int i = 0; i < 5; i++)
    {

        cin >> numbers[i];

        // Add each number to sum

        sum = sum + numbers[i];

    }
    // Display total sum

    cout << "Sum of all numbers = " << sum << endl;

    return 0;
}
```

Output:

```
Enter 5 numbers:
1
2
3
4
5
Sum of all numbers = 15
```

Lab 08

Two Dimission Array in C++

Learning Objectives:

The objective of this lab is to understand and apply two-dimensional arrays in C++.

Students will

learn how to:

- Declare and initialize 2D arrays in C++.
- Access and display elements of a 2D array using nested loops.

- Perform row-wise and column-wise sum calculations on 2D arrays.
- Find the maximum value in a 2D array.

Introduction:

The simplest form of a multidimensional array is the two-dimensional (2D) array. A two-dimensional array is essentially a list of one-dimensional arrays arranged in rows and columns, similar to a matrix or table.

To declare a 2D array:

Syntax:

```
type arrayName[rows][columns];
```

Example: `int matrix[3][4];` declares a 2D integer array with 3 rows and 4 columns.

To initialize a 2D array:

```
int a[3][4] = {
    {0, 1, 2, 3},
    // row 0
    {4, 5, 6, 7},
    // row 1
    {8, 9, 10, 11} // row 2
};
```

Elements are accessed using two indices: `a[row][column]`. Nested for loops are typically used to iterate over all elements.

Task 1:

Two-Dimensional Array Declaration and Initialization

Declare a 3x3 integer array called 'matrix', initialize it with values 1 through 9, and display all elements using nested loops.

Code:

```
#include <iostream>
using namespace std;
int main()
{
    // 3x3 ka 2D array declare aur initialize kar rahe hain

    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    cout << "Matrix ke elements hain:\n";

    // Nested loop se matrix ke tamam elements display karenge

    for (int i = 0; i < 3; i++)
```

```
{  
for (int j = 0; j < 3; j++)  
{  
// Har element print karna  
cout << matrix[i][j] << " ";  
}  
// Har row ke baad next line  
cout << endl;  
}  
return 0;  
}
```

Output:

```
Matrix ke elements hain:  
1 2 3  
4 5 6  
7 8 9
```

Task 2:

Sum of Rows and Columns in a Two-Dimensional Array

Declare and initialize a 3x3 integer array with random values. Calculate and display the sum of each row, each column, and the total sum of all elements.

Code:

```
#include <iostream>  
using namespace std;  
int main()  
{  
// 3x3 array declare aur initialize  
  
int matrix[3][3] = {  
  
    {2, 4, 6},  
    {1, 3, 5},  
    {7, 8, 9}  
};  
  
int totalSum = 0;  
  
// Rows ka sum nikalna  
  
cout << "Rows ka sum:\n";  
  
for (int i = 0; i < 3; i++)  
{  
    int rowSum = 0;  
  
    for (int j = 0; j < 3; j++)
```

```
{
rowSum = rowSum + matrix[i][j];

totalSum = totalSum + matrix[i][j];
}
cout << "Row " << i + 1 << " ka sum = " << rowSum << endl;

}
cout << endl;

// Columns ka sum nikalna

cout << "Columns ka sum:\n";

for (int j = 0; j < 3; j++)

{
int colSum = 0;

for (int i = 0; i < 3; i++)
{
colSum = colSum + matrix[i][j];
}
cout << "Column " << j + 1 << " ka sum = " << colSum << endl;

}
cout << endl;

// Total sum display karna

cout << "Tamam elements ka total sum = " << totalSum << endl;

return 0;
}
```

Output:

```
Rows ka sum:
Row 1 ka sum = 12
Row 2 ka sum = 9
Row 3 ka sum = 24
Columns ka sum:
Column 1 ka sum = 10
Column 2 ka sum = 15
Column 3 ka sum = 20
Tamam elements ka total sum = 45
```

Task 3:

Finding the Maximum Value in a Two-Dimensional Array

Declare and initialize a 4x4 integer array with random values. Find and display the maximum value in the entire array.

Code:

```
#include <iostream>
```

```
using namespace std;
int main()
{
    // 4x4 array declare aur initialize

    int matrix[4][4] = {
        {12, 25, 7, 18},
        {30, 14, 9, 22},
        {5, 17, 40, 11},
        {8, 19, 13, 27}
    };

    // Pehla element maximum maan lete hain

    int max = matrix[0][0];

    // Nested loop se maximum value find karenge

    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            // Agar current element bara ho to max update karo

            if (matrix[i][j] > max)
            {
                max = matrix[i][j];
            }
        }
    }

    // Maximum value display karna

    cout << "Matrix ki maximum value = " << max << endl;
    return 0;
}
```

Output:

```
Matrix ki maximum value = 40
```

Lab 09

Introduction to in C++

Objectives

By the end of this lab, students were able to:

- Understand what functions are and why they are used in C++.
- Declare function prototypes before the main() function.
- Define and call functions with zero, one, and multiple parameters.
- Work with functions that return values and those that do not (void).

Background / Theory

Functions allow a long program to be divided into named, reusable sections. A C++ function has a name, may accept parameters, and may return a result. The prototype of a function declares its return type, name, and parameter types so the compiler recognises calls before seeing the full definition.

The general syntax of a function prototype is:

```
returnType functionName(paramType1, paramType2, ...);
```

Functions we have used throughout the course, such as main(), follow this same pattern. void before a function name means it returns nothing; int, float, etc. mean it returns a value of that type.

Task 1: Function Introduction and Declaration

Problem Statement

Declare a function called "greet" that takes no parameters and returns nothing. Inside the function, display a greeting message of your choice. Call the function from the main program.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: greet takes no parameters and returns nothing
void greet();

int main()
{
    // Calling the greet function from main
    greet();
    return 0;
}

// Function definition: displays a custom greeting message
void greet()
{
    cout << "Welcome to EE-170L Computer Programming Lab!" << endl;
    cout << "We are learning Functions in C++ today." << endl;
}
```

Explanation

- The prototype `void greet();` tells the compiler about the function before `main()`.
- `void` means the function returns no value; the empty parentheses mean no parameters.
- `greet()` is called inside `main()` — execution jumps to the function body, prints the messages, then returns to `main()`.

Expected Output

```
Welcome to EE-170L Computer Programming Lab!
We are learning Functions in C++ today.
```

Task 2: Basic Function Declaration and Call

Problem Statement

Declare a function called "sayHello" that takes no parameters and returns nothing. Inside the function, display the message "Hello, World!". Call the function from the main program.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: sayHello takes no parameters and returns nothing
void sayHello();

int main()
{
    // Calling the sayHello function from main
    sayHello();
    return 0;
}

// Function definition: prints the classic Hello World message
void sayHello()
{
    cout << "Hello, World!" << endl;
}
```

Explanation

- `sayHello()` has an empty parameter list and `void` return type — identical in structure to `greet()` in Task 1.
- This task demonstrates the most basic form of a user-defined function.
- The function call in `main()` transfers control to the function body, which executes `cout` and then returns.

Expected Output

Hello, World!

Task 3: Array Sum and Average Using a Function

Problem Statement

Write a C++ program that takes 10 integers from the user into a 1D array, uses a function `calculateSum()` to find the sum, and then calculates and displays the average in `main()`.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: accepts array and its size, returns the sum
int calculateSum(int arr[], int size);

int main()
{
    const int SIZE = 10;           // Size of the array
    int numbers[SIZE];             // 1D array to store 10 integers
    double average;                // Variable to store the average

    // Taking 10 integer inputs from the user
    cout << "Enter 10 integer values:" << endl;
    for (int i = 0; i < SIZE; i++)
    {
        cout << "Element [" << i + 1 << "]: ";
        cin >> numbers[i];
    }

    // Calling calculateSum function and storing the result
    int sum = calculateSum(numbers, SIZE);

    // Calculating the average using the returned sum
    average = (double)sum / SIZE;

    // Displaying the results
    cout << "Sum of all elements: " << sum << endl;
    cout << "Average of all elements: " << average << endl;

    return 0;
}

// Function definition: iterates through array and accumulates sum
int calculateSum(int arr[], int size)
{
    int total = 0;                 // Variable to accumulate sum
    for (int i = 0; i < size; i++)
    {
        total += arr[i];           // Add each element to total
    }
}
```

```
}  
    return total;           // Return the final sum  
}
```

Explanation

- The array `numbers[SIZE]` is declared in `main()` and filled via a for loop with user input.
- `calculateSum(int arr[], int size)` accepts the array and its size, iterates through all elements, and returns their total.
- `main()` calls the function, stores the returned sum, then computes the average as a double to preserve decimal precision.
- Using `(double)sum` casts the integer to a double before division, preventing integer truncation.

Sample Output

```
Enter 10 integer values:  
Element [1]: 5  
Element [2]: 10  
...  
Element [10]: 15  
Sum of all elements: 100  
Average of all elements: 10
```

Lab 10

More About Function in C++

Objectives

By the end of this lab, students were able to:

- Understand the difference between call by value and call by reference.
- Modify original variables using reference parameters (& operator).
- Use default parameter values in function declarations.
- Write functions that accept parameters and return computed values.

Background / Theory

Call by Value

When a function is called by value, a copy of the argument is made inside the function. Changes to the copy do not affect the original variable in the caller. This is the default behaviour in C++.

Call by Reference

When a function is called by reference (using & in the parameter list), no copy is made — the function works directly with the caller's variable. Any change inside the function is reflected in the caller. This is useful when you need to modify the original variable or return multiple values.

Default Parameter Values

A function parameter can be given a default value in its declaration. If the caller omits that argument, the default is used. If the caller provides a value, the default is ignored.

Task 1: Function with Parameters

Problem Statement

Declare a function called "multiply" that takes two integer parameters num1 and num2. Calculate and display the product inside the function. Call the function from main with two numbers.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: multiply takes two integers and returns nothing
void multiply(int num1, int num2);

int main()
{
    int a, b;

    // Taking two numbers as input from the user
    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter second number: ";
    cin >> b;

    // Calling multiply function with two arguments
    multiply(a, b);

    return 0;
}

// Function definition: computes and displays the product of two numbers
void multiply(int num1, int num2)
{
    int product = num1 * num2; // Calculate the product
    cout << "Product of " << num1 << " and " << num2;
    cout << " = " << product << endl;
}
```

Explanation

- The prototype `void multiply(int num1, int num2);` declares two integer parameters and no return value.
- The function computes the product locally and prints it — the result is displayed inside the function, not returned.
- This illustrates pass-by-value: `num1` and `num2` inside `multiply` are local copies of `a` and `b` from `main()`.

Sample Output

```
Enter first number: 6
Enter second number: 7
Product of 6 and 7 = 42
```

Task 2: Function with Return Value

Problem Statement

Declare a function called "getSquare" that takes an integer parameter "number", calculates its square, returns the result, and displays it in main.

Source Code

```
#include <iostream>
using namespace std;

// Function prototype: getSquare takes an integer and returns its square
int getSquare(int number);

int main()
{
    int num;

    // Taking a number as input from the user
    cout << "Enter a number: ";
    cin >> num;

    // Calling getSquare and storing the returned value
    int result = getSquare(num);

    // Displaying the square of the number
    cout << "Square of " << num << " = " << result << endl;

    return 0;
}

// Function definition: calculates and returns the square of a number
int getSquare(int number)
{
    return number * number;    // Multiply number by itself and return
}
```

Explanation

- The prototype `int getSquare(int number);` declares that the function accepts one integer and returns an integer.
- `number * number` computes the square; the return statement sends this value back to the caller.

- In main(), the returned value is stored in result and then printed — this separates computation from display.

Sample Output

```
Enter a number: 9
Square of 9 = 81
```

Conclusion

Lab 09 introduced the fundamentals of user-defined functions in C++: writing prototypes, defining functions with varying numbers of parameters, and returning values. Lab 10 extended this knowledge by exploring call by value (where function changes do not affect the caller's variables) versus call by reference (where they do), and default parameter values.

Across all five tasks, comments were used consistently to explain prototypes, function calls, and definitions, forming good documentation habits. Functions are a core tool for writing clean, modular, and reusable C++ programs.

Lab 11

String in C++

1 Objective

The objective of this lab session is to introduce and apply the following concepts in C++:

- C-String (C-style strings)
- Declaration and Initialization of C-Strings
- String Objects using the Standard C++ Library

2 Introduction to C++ Strings

A string is a collection of characters. There are two types of strings commonly used in C++:

- C-strings (C-style Strings)
- Strings that are objects of the string class (Standard C++ Library)

2.1 C-Strings

In C programming, the collection of characters is stored in the form of arrays. This is also supported in C++, hence the name C-strings. C-strings are arrays of type char terminated with a null character (\0). The ASCII value of the null character is 0.

2.1.1 Declaration and Initialization

The following examples demonstrate the various ways to declare and initialize a C-string:

```
char myString[] = "Hello";           // Automatically adds \0
char myString[6] = "Hello";         // Explicit size
char myString[] = {'H','e','l','l','o','\0'}; // Character array
char myString[100] = "Hello";       // Pre-allocating extra space
```

Note: The C++ compiler automatically places `\0` at the end of the string constant. Unlike arrays, it is not necessary to use all the allocated space.

2.2 Reading Strings

Example 1: Basic string input with cin

```
#include <iostream>
using namespace std;
int main() {
    char str[100];
    cout << "Enter a string: ";
    cin >> str;
    cout << "You entered: " << str << endl;
    cout << "Enter another string: ";
    cin >> str;
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output

```
Enter a string: Department
You entered: Department
Enter another string: Electrical Engineering
You entered: Electrical
```

Notice: The extraction operator `>>` with `cin` considers a space " " as a terminating character. Hence, only "Electrical" is displayed instead of "Electrical Engineering".

Example 2: Reading a full line with cin.get()

```
#include <iostream>
using namespace std;
int main() {
    char str[100];
    cout << "Enter a string: ";
    cin.get(str, 100);
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output

```
Enter a string: Electrical Engineering
```

```
You entered: Electrical Engineering
```

The `cin.get()` function takes two arguments: the name of the string (address of first element) and the maximum size of the array. This allows reading strings containing blank spaces.

2.3 String Objects

In C++, you can also create a string object for holding strings. Unlike char arrays, string objects have no fixed length and can be extended as required.

```
#include <iostream>
using namespace std;
int main() {
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output

```
Enter a string: University of Engineering and Technology
```

```
You entered: University of Engineering and Technology
```

The `getline()` function takes the input stream (`cin`) as the first parameter and the string variable as the second, allowing full-line input including spaces.

3 String Functions

C++ supports a wide range of functions that manipulate null-terminated strings. The table below summarizes the most commonly used ones:

Function	Example	Description
<code>strcpy()</code>	<code>strcpy(s1, s2);</code>	Copies string <code>s2</code> into string <code>s1</code> .
<code>strcat()</code>	<code>strcat(s1, s2);</code>	Concatenates string <code>s2</code> onto the end of string <code>s1</code> .
<code>strlen()</code>	<code>strlen(s);</code>	Returns the length of string <code>s</code> .
<code>strcmp()</code>	<code>strcmp(s1, s2);</code>	Returns 0 if <code>s1==s2</code> ; <0 if <code>s1<s2</code> ; >0 if <code>s1>s2</code> .
<code>strrev()</code>	<code>strrev(s);</code>	Reverses the given string.
<code>strlwr()</code>	<code>strlwr(s);</code>	Converts string to lowercase.
<code>strupr()</code>	<code>strupr(s);</code>	Converts string to uppercase.

4 Lab Tasks — Lab 11

Task 1: String Length Calculation

Write a C++ program that declares and initializes a string variable called message with a phrase of your choice and uses a built-in function to calculate and display the length of the string.

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char message[] = "UET Peshawar - Engineering Excellence";
    int len = strlen(message);
    cout << "Message: " << message << endl;
    cout << "Length of the string: " << len << endl;
    return 0;
}
```

Output

```
Message: UET Peshawar - Engineering Excellence
Length of the string: 37
```

Task 2: String Concatenation

Declare two string variables called firstName and lastName, initialize them with your first and last name, concatenate them to form a full name, and display it.

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char firstName[50] = "Ali";
    char lastName[50] = "Khan";
    char fullName[100];
    strcpy(fullName, firstName);
    strcat(fullName, " ");
    strcat(fullName, lastName);
    cout << "First Name: " << firstName << endl;
    cout << "Last Name: " << lastName << endl;
    cout << "Full Name: " << fullName << endl;
    return 0;
}
```

Output

```
First Name: Ali
Last Name: Khan
Full Name: Ali Khan
```

Lab 12

Pointers in C++

1 Objective

The objective of this lab session is to introduce and apply the following concepts in C++:

- Computer Memory and Variables
- Pointers
- Pointer Declaration
- Referencing Operator (&)
- Dereferencing Operator (*)

2 Computer Memory

The computer's memory is made up of bytes. Each byte has a unique address associated with it. When a program runs, variables are stored in memory at specific addresses determined by the compiler. The size of the memory block reserved depends on the data type: for example, on a 32-bit PC an integer occupies 4 bytes, while on older 16-bit PCs integers occupied 2 bytes.

2.1 Variables and Memory

When a variable is declared, the compiler reserves a specific block of memory to hold its value. Two distinct steps occur when a variable is accessed:

- The program locates the address reserved for the variable.
- The contents stored at that address are retrieved.

```
#include <iostream>
using namespace std;
int main() {
    float fl = 3.14; // declare a variable of type float
    cout << fl;
    return 0;
}
```

Output

3.14

Note: The float value 3.14 is actually stored as 32 ones and zeros (4 bytes × 8 bits) in memory, not as individual characters.

3 Pointers in C++

A pointer is a variable that holds the memory address of another variable. Unlike regular variables that store values directly, a pointer stores the address where a value is kept. Because a pointer holds an address rather than a value, it has two components:

- The pointer variable itself (holds the address).
- The value stored at that address.

3.1 Pointer Declaration

The syntax for declaring a pointer variable is:

```
dataType *variableName;

// Examples:
int *ptr;      // pointer to an integer
float *fptr;   // pointer to a float
char *cptr;    // pointer to a char
```

The asterisk (*) tells the compiler that a pointer variable is being created. The data type specifies the type of data at the address the pointer will hold.

3.2 Referencing Operator (&)

The address-of operator (&) retrieves the memory address of a variable and assigns it to a pointer:

```
int *ptr;
int k;
ptr = &k;    // ptr now holds the address of k
```

After this statement, ptr is said to "point to" k.

3.3 Dereferencing Operator (*)

The dereference operator (*) accesses the value at the address stored in a pointer:

```
*ptr = 7;          // sets the value of k to 7 (via pointer)
cout << *ptr;      // prints the value stored at address ptr
```

When * is used in this context, we are referring to the value that ptr points to, not the pointer's own address value.

3.4 Example: Pointer Operations

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int b = 15;
    int *p1, *p2; // declare two pointers
    p1 = &a;      // p1 points to a
    p2 = &b;      // p2 points to b
    *p1 = 10;     // a = 10
    *p2 = *p1;    // b = 10 (value at p1)
```

```
*p1 = 20;          // a = 20
cout << "a = " << a << endl;
cout << "b = " << b << endl;
return 0;
}
```

Output

```
a = 20
b = 10
```

4 Lab Tasks — Lab 12

Task 1: Pointer Declaration and Initialization

Declare an integer variable num, a pointer ptr, initialize the pointer to point to num, and display the values using the dereference operator.

```
#include <iostream>
using namespace std;
int main() {
    int num = 42;
    int *ptr;
    ptr = &num;
    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Value of ptr (address): " << ptr << endl;
    cout << "Value pointed by ptr: " << *ptr << endl;
    return 0;
}
```

Output

```
Value of num: 42
Address of num: 0x61ff08
Value of ptr (address): 0x61ff08
Value pointed by ptr: 42
```

Task 2: Pointer Arithmetic

Declare an integer array, use a pointer to traverse and display its elements using pointer arithmetic.

```
#include <iostream>
using namespace std;
int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int *ptr = numbers; // pointer to first element
    int size = 5;
```

```
cout << "Array elements using pointer arithmetic:" << endl;
for (int i = 0; i < size; i++) {
    cout << "numbers[" << i << "] = " << *(ptr + i) << endl;
}
return 0;
}
```

Output

Array elements using pointer arithmetic:

numbers[0] = 10

numbers[1] = 20

numbers[2] = 30

numbers[3] = 40

numbers[4] = 50

Laboratory Session 13

Structures in C++

1 Objective

The objective of this lab session is to introduce and apply the following concepts in C++:

- Defining and declaring structures
- Accessing structure members using dot notation
- Initializing structure variables
- Copying structure variables
- Using structures to model real-world entities

2 Introduction to Structures

A structure is a collection of variables of different data types grouped under a single name. Unlike arrays, where all elements must be of the same type, a structure can hold members of mixed types (int, float, char, etc.). The data items in a structure are called its members.

Key characteristics of structures:

- Members can be of different data types.
- A structure type declaration defines the form of the structure, not memory allocation.
- Memory is allocated only when a structure variable is declared.
- The closing brace in the structure declaration must be followed by a semicolon.
- Structures can be nested within other structures.
- Structure variables can be passed to functions either individually or as a whole.

2.1 Syntax

```
struct StructureName {
    dataType member1;
    dataType member2;
    // ...
};

// Example:
struct part {
    int modelnumber;    // ID number of car
    int partnumber;    // ID number of car part
    float cost;        // cost of part
};
```

2.2 Example: One Structure Variable

```
#include <iostream>
using namespace std;
struct part {
    int modelnumber;
    int partnumber;
    float cost;
};
int main() {
    part part1;
    part1.modelnumber = 6244;
    part1.partnumber = 373;
    part1.cost = 217.55F;
    cout << "Model : " << part1.modelnumber << endl;
    cout << "Part  : " << part1.partnumber << endl;
    cout << "Cost  : " << part1.cost << " $" << endl;
    return 0;
}
```

Output

```
Model : 6244
Part  : 373
Cost  : 217.55 $
```

2.3 Example: Two Structure Variables (Copy)

Structure variables can be directly assigned to one another, which performs a member-wise copy:

```
part part1 = {6244, 373, 217.55F};
part part2;
part2 = part1;    // copies all members from part1 to part2
```

Output

```
Model : 6244
Part  : 373
Cost  : 217.55 $
Model : 6244
Part  : 373
Cost  : 217.55 $
```

2.4 Example: Employee Information

```
#include <iostream>
using namespace std;
struct Person {
    char name[50];
    int age;
    float salary;
```

```
};  
int main() {  
    Person p1;  
    cout << "Enter Full name: ";  
    cin.get(p1.name, 50);  
    cout << "Enter age: ";  
    cin >> p1.age;  
    cout << "Enter salary: ";  
    cin >> p1.salary;  
    cout << "\nDisplaying Information." << endl;  
    cout << "Name: " << p1.name << endl;  
    cout << "Age: " << p1.age << endl;  
    cout << "Salary: " << p1.salary;  
    return 0;  
}
```

Output

Enter Full name: Murad Ali

Enter age: 34

Enter salary: 1200

Displaying Information.

Name: Murad Ali

Age: 34

Salary: 1200

3 Lab Tasks — Lab 13

Task 1: Person Structure

Declare a structure called Person with members name (string), age (integer), and address (string). Initialize and display using dot notation.

```
#include <iostream>  
#include <string>  
using namespace std;  
struct Person {  
    string name;  
    int age;  
    string address;  
};  
int main() {  
    Person person1;  
    person1.name = "Ahmed Khan";  
    person1.age = 21;  
    person1.address = "House 12, Street 4, Peshawar";  
    cout << "Name: " << person1.name << endl;
```

```
cout << "Age: " << person1.age << endl;
cout << "Address: " << person1.address << endl;
return 0;
}
```

Output

Name: Ahmed Khan

Age: 21

Address: House 12, Street 4, Peshawar

Task 2: Student Structure

Declare a structure called Student with members name (string), age (integer), and grade (char). Initialize and display the members.

```
#include <iostream>
#include <string>
using namespace std;
struct Student {
    string name;
    int age;
    char grade;
};
int main() {
    Student student1;
    student1.name = "Sara Malik";
    student1.age = 20;
    student1.grade = 'A';
    cout << "Student Name: " << student1.name << endl;
    cout << "Age: " << student1.age << endl;
    cout << "Grade: " << student1.grade << endl;
    return 0;
}
```

Output

Student Name: Sara Malik

Age: 20

Grade: A

Task 3: Book Structure

Declare a structure called Book with members title (string), author (string), price (float), and pages (integer). Declare two variables book1 and book2, initialize and display both.

```
#include <iostream>
#include <string>
using namespace std;
```

```
struct Book {
    string title;
    string author;
    float price;
    int pages;
};

int main() {
    Book book1 = {"C++ Primer", "Stanley Lippman", 45.99f, 975};
    Book book2 = {"The C Programming Language", "K&R", 35.50f, 272};
    cout << "--- Book 1 ---" << endl;
    cout << "Title: " << book1.title << endl;
    cout << "Author: " << book1.author << endl;
    cout << "Price: $" << book1.price << endl;
    cout << "Pages: " << book1.pages << endl;
    cout << "--- Book 2 ---" << endl;
    cout << "Title: " << book2.title << endl;
    cout << "Author: " << book2.author << endl;
    cout << "Price: $" << book2.price << endl;
    cout << "Pages: " << book2.pages << endl;
    return 0;
}
```

Output

```
--- Book 1 ---
Title: C++ Primer
Author: Stanley Lippman
Price: $45.99
Pages: 975
--- Book 2 ---
Title: The C Programming Language
Author: K&R
Price: $35.50
Pages: 272
```

1 Objective:

The objective of this lab session is to introduce and apply the following concepts in C++:

- Defining and declaring structures
- Accessing structure members using dot notation
- Initializing structure variables
- Copying structure variables
- Using structures to model real-world entities

2 Introduction to Structures:

A structure is a collection of variables of different data types grouped under a single name. Unlike arrays, where all elements must be of the same type, a structure can hold members of mixed types (int, float, char, etc.). The data items in a structure are called its members.

Key characteristics of structures:

- Members can be of different data types.
- A structure type declaration defines the form of the structure, not memory allocation.
- Memory is allocated only when a structure variable is declared.
- The closing brace in the structure declaration must be followed by a semicolon.
- Structures can be nested within other structures.
- Structure variables can be passed to functions either individually or as a whole.

2.1 Syntax:

```
struct StructureName {           // Define structure template
    dataType member1;           // First member
    dataType member2;           // Second member
    // ...
};

// Example structure definition:
struct part {
    int modelnumber;             // ID number of car
    int partnumber;             // ID number of car part
    float cost;                  // cost of part
};
```

2.2 Example: One Structure Variable:

```
#include <iostream> // for input/output operations
using namespace std;
// Declare the part structure
struct part {
    int modelnumber;
    int partnumber;
    float cost;
};
int main() {
    // Declare a structure variable of type part
    part part1;
    // Initialize structure members using dot notation
    part1.modelnumber = 6244;
    part1.partnumber = 373;
    part1.cost = 217.55F;
    // Display structure members using dot notation
    cout << "Model : " << part1.modelnumber << endl;
```

```
cout << "Part   : " << part1.partnumber << endl;
cout << "Cost   : " << part1.cost << " $" << endl;
return 0;
}
```

Output

```
Model : 6244
Part   : 373
Cost   : 217.55 $
```

2.3 Example: Two Structure Variables (Copy):

Structure variables can be directly assigned to one another, which performs a member-wise copy:

```
// Declare and initialize book1 using initializer list
part part1 = {6244, 373, 217.55F};
// Declare a second structure variable
part part2;
// Copy all members from part1 to part2
part2 = part1;
```

Output

```
Model : 6244
Part   : 373
Cost   : 217.55 $
Model : 6244
Part   : 373
Cost   : 217.55 $
```

2.4 Example: Employee Information:

```
#include <iostream> // for input/output operations
using namespace std;
// Declare the Person structure
struct Person {
    char name[50];
    int age;
    float salary;
};
int main() {
    // Declare a structure variable of type Person
    Person p1;
    // Prompt and read full name (up to 50 characters)
    cout << "Enter Full name: ";
    cin.get(p1.name, 50);
    // Prompt and read age
    cout << "Enter age: ";
    cin >> p1.age;
```

```
// Prompt and read salary
cout << "Enter salary: ";
cin >> p1.salary;

// Display all information
cout << "\nDisplaying Information." << endl;
cout << "Name: " << p1.name << endl;
cout << "Age: " << p1.age << endl;
cout << "Salary: " << p1.salary;
return 0;
}
```

Output

Enter Full name: Murad Ali

Enter age: 34

Enter salary: 1200

Displaying Information.

Name: Murad Ali

Age: 34

Salary: 1200

3 Lab Tasks :

Task 1: Person Structure:

Declare a structure called Person with members name (string), age (integer), and address (string). Initialize and display using dot notation.

```
#include <iostream> // for input/output operations
#include <string>    // for string class
using namespace std;

// Declare the Person structure
struct Person {
    string name;
    int age;
    string address;
};

int main() {
    // Declare a structure variable of type Person
    Person person1;

    // Initialize person1 members using dot notation
    person1.name = "Ahmed Khan";
    person1.age = 21;
    person1.address = "House 12, Street 4, Peshawar";

    // Display person1 members using dot notation
    cout << "Name: " << person1.name << endl;
    cout << "Age: " << person1.age << endl;
}
```

```
cout << "Address: " << person1.address << endl;
return 0;
}
```

Output

Name: Ahmed Khan

Age: 21

Address: House 12, Street 4, Peshawar

Task 2: Student Structure:

Declare a structure called Student with members name (string), age (integer), and grade (char). Initialize and display the members.

```
#include <iostream> // for input/output operations
#include <string> // for string class
using namespace std;
// Declare the Student structure
struct Student {
    string name;
    int age;
    char grade;
};
int main() {
    // Declare a structure variable of type Student
    Student student1;
    // Initialize student1 members using dot notation
    student1.name = "Sara Malik";
    student1.age = 20;
    student1.grade = 'A';
    // Display student1 members
    cout << "Student Name: " << student1.name << endl;
    cout << "Age: " << student1.age << endl;
    cout << "Grade: " << student1.grade << endl;
    return 0;
}
```

Output

Student Name: Sara Malik

Age: 20

Grade: A

Task 3: Book Structure:

Declare a structure called Book with members title (string), author (string), price (float), and pages (integer). Declare two variables book1 and book2, initialize and display both.

```
#include <iostream> // for input/output operations
#include <string> // for string class
using namespace std;
// Declare the Book structure
struct Book {
    string title;
    string author;
    float price;
    int pages;
};
int main() {
    // Declare and initialize book1 using initializer list
    Book book1 = {"C++ Primer", "Stanley Lippman", 45.99f, 975};
    // Declare and initialize book2 using initializer list
    Book book2 = {"The C Programming Language", "K&R", 35.50f, 272};
    // Display book1 information
    cout << "--- Book 1 ---" << endl;
    cout << "Title: " << book1.title << endl;
    cout << "Author: " << book1.author << endl;
    cout << "Price: $" << book1.price << endl;
    cout << "Pages: " << book1.pages << endl;
    // Display book2 information
    cout << "--- Book 2 ---" << endl;
    cout << "Title: " << book2.title << endl;
    cout << "Author: " << book2.author << endl;
    cout << "Price: $" << book2.price << endl;
    cout << "Pages: " << book2.pages << endl;
    return 0;
}
```

Output

```
--- Book 1 ---
Title: C++ Primer
Author: Stanley Lippman
Price: $45.99
Pages: 975
--- Book 2 ---
Title: The C Programming Language
Author: K&R
Price: $35.50
Pages: 272
```

Lab 14(OEL)
Opeing Ended Lab
Grading system

1. Objective:

The objective of this lab project is to design and implement a Relative Grading System in C++ that manages student records across multiple papers/subjects. The system assigns grades based on the relative performance of students within the same paper, computes per-student GPA and CGPA, and generates a Detailed Marks Certificate (DMC) transcript. The project demonstrates the use of structures, arrays, functions, sorting, and input validation in C++.

2. Tools & Environment:

Tool / Component	Details
Programming Language	C++ (Standard: C++11 or later)
Compiler	g++ / MinGW / MSVC
IDE	Visual Studio Code / Code::Blocks / Dev-C++
Libraries Used	<iostream>, <iomanip>, <cmath>, <string>
Platform	Windows / Linux / macOS

3. Theory & Background:

3.1 Relative Grading

Relative grading (also called norm-referenced grading) ranks students against each other rather than against a fixed absolute scale. The top performers receive the highest grades regardless of the raw marks. This approach accounts for variation in exam difficulty and ensures a consistent grade distribution across different semesters.

3.2 GPA Scale Used

Grade	GPA Points	Rank Position	Students
A	4.0	Top 10	Positions 1–10
A-	3.7	11–15	Positions 11–15
B+	3.3	16–20	Positions 16–20
B	3.0	21–25	Positions 21–25
B-	2.7	26–30	Positions 26–30
C+	2.3	31–35	Positions 31–35
C	2.0	36–40	Positions 36–40
D	1.0	41–45	Positions 41–45
F	0.0	46+	All remaining

3.3 CGPA Calculation:

CGPA is the weighted average of GPA points across all enrolled papers, calculated as:

$$\text{CGPA} = \frac{\sum(\text{GPA} \times \text{Credit Hours})}{\sum(\text{Credit Hours})}$$

4. Program Description:

The program is a console-based Relative Grading System that allows a teacher or administrator to manage marks for multiple papers and students. The main functional areas are:

- Enter paper details including student marks
- Update existing student records
- View per-paper result sheets with topper
- Generate a DMC (Detailed Marks Certificate) for any student

4.1 Data Structures:

Structure / Array	Type	Purpose
Student	struct	Holds roll number and name
PaperRecord	struct	Holds student, paper name, credit hours, marks, grade, GPA
records[]	PaperRecord[500]	Global array storing all student-paper entries
paperNames[]	string[10]	Stores unique paper names entered so far
totalRecords	int	Count of all records stored
totalPapers	int	Count of unique papers registered

4.2 Functions Overview

Function	Description
paperExists()	Checks if a paper name already exists in paperNames[]
addPaper()	Adds a new paper name to the global list if not duplicate
duplicateRecord()	Checks for duplicate (rollNo, paper) combination
assignGrades()	Sorts students of one paper by marks and assigns relative grades
showTopper()	Finds and displays the top-marks student for a given paper
enterDetails()	Prompts for paper info and student marks, validates inputs
updateDetails()	Finds a record by roll no. and paper, allows update of name/marks/roll
paperDetailsMenu()	Sub-menu: Enter or Update paper details
seePaperResults()	Lists all papers, lets user pick one, displays formatted result table
showDMC()	Builds unique student list, shows full DMC with CGPA for selected student
main()	Entry point; runs do-while loop with 4-option main menu

5. Code Walkthrough

5.1 Structures

Two nested structures are defined:

```
struct Student {
    string rollNo;
```

```

        string name;
    };

    struct PaperRecord {
        Student student;    // nested struct
        string paperName;
        int    creditHours;
        float  marks;
        string grade;
        float  gpa;
    };

```

5.2 Relative Grade Assignment (assignGrades)

The function collects indexes of all records for a given paper, sorts them in descending order of marks using a simple selection sort on the index array, then assigns grades according to rank position:

```

// Sort indexes descending by marks
for (int i = 0; i < count - 1; i++)
    for (int j = i+1; j < count; j++)
        if (records[indexes[j]].marks > records[indexes[i]].marks)
            swap(indexes[i], indexes[j]);

// Assign grades by rank
if (i < 10)    grade = "A",    gpa = 4.0;
else if (i < 15) grade = "A-",  gpa = 3.7;
else if (i < 20) grade = "B+",  gpa = 3.3;
// ... and so on

```

5.3 Input Validation

```

while (temp.marks < 0 || temp.marks > 100) {
    cout << "Invalid Marks. Enter Again: ";
    cin  >> temp.marks;
}

```

5.4 CGPA Calculation (showDMC)

```

float totalPoints = 0;
int   totalCH     = 0;

for (int i = 0; i < totalRecords; i++) {
    if (records[i].student.rollNo == selectedRoll) {
        totalPoints += records[i].gpa * records[i].creditHours;
        totalCH     += records[i].creditHours;
    }
}
float cgpa = totalPoints / totalCH;

```

6. Program Flow

The program begins at main() and enters a do-while loop presenting the main menu. The control flow for each option is summarised below:

Option	Flow
1	paperDetailsMenu() → Enter Details: validate paper, loop through students, validate marks, store record, call assignGrades() OR Update Details: locate record, offer

	name/marks/roll update, re-grade if marks changed
2	seePaperResults(): list papers → user picks → print formatted table of all students in that paper → showTopper()
3	showDMC(): build unique student list → user picks → print all papers for that student with CGPA
4	Exit do-while loop, program ends

7. Sample Output

7.1 Main Menu

```
=====
      RELATIVE GRADING SYSTEM
=====
1. Enter Paper Details
2. See Each Paper Results
3. See DMC Transcript
4. Exit
Enter Choice:
```

7.2 Paper Result Table

```
===== Mathematics RESULT =====
Roll No      Name      Marks      Grade      GPA
2023-CS-10    Ali        92         A          4.0
2023-CS-05    Sara       88         A          4.0
2023-CS-17    Ahmed     85         A          4.0
...

===== TOPPER =====
Paper : Mathematics
Name  : Ali
Marks : 92
```

7.3 DMC Transcript

```
===== DMC =====
Name      : Ali
Roll No   : 2023-CS-10

Subject    CH      Marks      Grade      GPA
Mathematics 3       92         A          4.0
Physics     3       78         B+         3.3
English     2       70         B          3.0

Total Credit Hours: 8
CGPA: 3.54
```

8. Analysis & Discussion

8.1 Strengths

- Nested structures cleanly separate student identity from paper performance data.
- Relative grading is re-evaluated every time a record is added or marks are updated, ensuring consistency.
- Duplicate detection prevents the same student being entered twice for the same paper.

- Input validation ensures marks remain within the 0–100 range.
- showDMC() dynamically aggregates data without storing CGPA separately, reducing redundancy.

8.2 Limitations

- Data is not persisted to a file; all records are lost when the program exits.
- Student and paper name fields do not support spaces (cin stops at whitespace).
- The fixed arrays (MAX_RECORDS = 500, MAX_PAPERS = 10) impose hard limits.
- The grade bands are hardcoded for exactly 45+ students; smaller classes may result in very few students receiving failing grades or an unbalanced distribution.

8.3 Suggested Improvements

- Add file I/O (fstream) to save and load records between sessions.
- Use getline() for names to support multi-word inputs.
- Replace raw arrays with vectors for dynamic sizing.
- Implement a search/filter function by student name or roll number.
- Add an absolute minimum pass mark (e.g. 40%) before applying relative bands.

9. Conclusion

This lab project successfully implements a console-based Relative Grading System in C++. It demonstrates key programming concepts including nested structures, 1D arrays, sorting algorithms, modular function design, input validation, and formatted output with setw(). The system correctly computes per-paper grades and per-student CGPA, and provides a working DMC generation feature. The project serves as a practical application of C++ fundamentals to a real-world academic problem.

References

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.
2. Stanley B. Lippman, C++ Primer, 5th Edition, Addison-Wesley, 2012.
3. cppreference.com — C++ Standard Library Reference. [http](http://cppreference.com)